

HYPERPARAMETER TUNING GUIDE

Effect of Increase vs Decrease | Baseline Accuracy: 96.67% | What to Tweak for Improvement

HYPERPARAMETER	WHEN INCREASED	WHEN DECREASED
LEARNING RATE Current: 0.001 (Adam) HIGH IMPACT	INCREASE Faster convergence initially Larger weight update steps Risk: overshooting minimum Risk: loss oscillates or diverges	DECREASE Smoother, more stable training Finer weight adjustments Risk: very slow convergence Risk: stuck in local minima
BATCH SIZE Current: 32 MED IMPACT	INCREASE Smoother gradient estimates Faster training (fewer updates/epoch) Risk: sharp minima, worse generalization Risk: needs more GPU memory	DECREASE Noisier gradients (regularization effect) Can improve generalization Risk: slower wall-clock training time More weight updates per epoch
EPOCHS Current: 50 MED IMPACT	INCREASE More time to learn patterns May improve if underfitting Risk: overfitting (memorization) Fix: use Early Stopping callback	DECREASE Less training time Model may not converge fully Risk: underfitting Only reduce if already overfitting
DROPOUT RATE Current: 0.3 MED IMPACT	INCREASE Stronger regularization Reduces overfitting Fewer active neurons each pass Risk: underfitting if too high (>0.5)	DECREASE More neurons active during training Higher model capacity utilized Risk: overfitting At 0.0 = no dropout at all
HIDDEN UNITS L1 Current: 128 MED IMPACT	INCREASE More capacity to learn complex patterns Richer feature representations Risk: overfitting on small datasets More parameters = more compute	DECREASE Simpler model, faster training Better if current model overfits Risk: underfitting May lose representational power
HIDDEN UNITS L2 Current: 64 MED IMPACT	INCREASE Less compression of features Retains more information from L1 Risk: overfitting, less regularization Wider bottleneck	DECREASE Stronger compression (bottleneck) Forces network to keep only key features Acts as implicit regularization Risk: loses important information

CATEGORICAL HYPERPARAMETERS & TUNING STRATEGIES

HYPERPARAMETER	ALTERNATIVE OPTIONS	EFFECT ON TRAINING
OPTIMIZER Current: Adam HIGH IMPACT	INCREASE SGD: needs higher LR (0.01), momentum=0.9 SGD can find flatter, better generalizing minima RMSprop: good for RNNs, adaptive LR SGD+scheduler often beats Adam final acc	DECREASE Adam is already the adaptive default Rarely need to go simpler than Adam Plain SGD (no momentum) = hardest to tune Adagrad: LR decays too aggressively
ACTIVATION (Hidden) Current: ReLU LOW IMPACT	INCREASE LeakyReLU: fixes dying ReLU problem small slope for x<0 instead of 0 ELU: smoother, pushes mean toward zero LeakyReLU can marginally improve acc	DECREASE Sigmoid: vanishing gradients, slow training Tanh: better than sigmoid, still saturates NEVER go back to sigmoid for hidden Both are computationally expensive
ACTIVATION (Output) Current: Softmax LOW IMPACT	INCREASE No alternative for multi-class classification Softmax is the only correct choice here For binary: switch to sigmoid (1 output neuron) For regression: use linear (no activation)	DECREASE N/A — softmax is required for this task Using ReLU here = meaningless outputs Using sigmoid = independent probabilities, won't sum to 1 (use only for multi-label)
LOSS FUNCTION Current: sparse_categorical_CE LOW IMPACT	INCREASE categorical_CE: same math, needs one-hot labels y_train = to_categorical(y_train, 10) Label smoothing: softens targets (0.1) Label smoothing can improve generalization	DECREASE MSE for classification: poor gradients Don't use MSE for classification tasks Hinge loss: SVM-style, rare in deep learning Focal loss: helps with class imbalance
VALIDATION SPLIT Current: 0.2 LOW IMPACT	INCREASE More data reserved for monitoring Better overfitting detection Less data available for training Risk: underfitting due to small train set	DECREASE More training data available Can improve accuracy on small datasets Less reliable overfitting detection At 0.0 = no validation monitoring at all
NUMBER OF LAYERS Current: 2 hidden layers MED IMPACT	INCREASE Deeper hierarchical feature learning More levels of abstraction Risk: vanishing gradients without BatchNorm Risk: overfitting on small datasets	DECREASE Simpler, faster model 1 layer = single level of abstraction only Risk: can't learn complex patterns May severely underfit

ACCURACY IMPROVEMENT STRATEGY — FROM 96.67% TO HIGHER

A PRIORITY RANKING — WHAT TO TWEAK FIRST

PRIORITY 1

ADD CALLBACKS (Biggest bang for zero risk)

1. Add EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True)
WHY: Your model may be overfitting after epoch 30 but training continues until 50. This recovers the best epoch automatically.
2. Add ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)
WHY: Starts with fast learning, then automatically fine-tunes with smaller steps. Almost always improves final accuracy.
3. Add BatchNormalization() after each Dense layer (before activation)
WHY: Normalizes layer inputs, stabilizes training, allows higher learning rates, acts as mild regularization.

PRIORITY 2

TUNE NUMERICAL HYPERPARAMETERS

1. Learning Rate: try 0.0005, 0.001, 0.002, 0.005 — small changes here have the LARGEST effect
2. Batch Size: try 16 (more noise = regularization on small dataset) or 64 (smoother gradients)
3. Dropout: try 0.2 (less aggressive) since 96.67% suggests model may NOT be overfitting much
4. Architecture: try 256->128 (more capacity) or 128->128 (wider second layer)

PRIORITY 3

ADVANCED TECHNIQUES (Diminishing returns at 96%+)

1. Data Augmentation: rotate, shift, zoom digits — effectively increases dataset size
2. K-Fold Cross Validation: 5-fold gives more reliable accuracy estimate
3. Ensemble: train 3-5 models, average predictions — typically +0.5-1% improvement

B CONCRETE EXPERIMENTS TO TRY (Copy-Paste Ready)

1	Add EarlyStopping + ReduceLROnPlateau	Prevents late-epoch overfitting, auto-tunes LR	97.0 - 97.5%
2	Reduce dropout 0.3 -> 0.2	More capacity utilized (model not heavily overfitting)	96.8 - 97.2%
3	Batch size 32 -> 16	More updates/epoch, regularization from noise	96.8 - 97.0%
4	Add BatchNorm after each Dense layer	Stabilizes training, mild regularization	97.0 - 97.5%
5	Architecture 128->64 to 256->128	More representational capacity	96.5 - 97.5%
6	LR 0.001 -> 0.0005 + more epochs (100)	Slower but finer convergence	96.8 - 97.2%
7	Combine #1 + #2 + #4 together	Best combined strategy	97.2 - 97.8%

KEY INSIGHT FOR YOUR PRESENTATION

At 96.67%, the model is already strong. The biggest gains come from CALLBACKS (EarlyStopping + LR Scheduler), not from changing architecture. Learning Rate is the single most sensitive hyperparameter — small changes cause large effects. Dropout at 0.3 may be slightly aggressive for a dataset this small — try 0.2. The optimal strategy: add callbacks first, then fine-tune LR, then reduce dropout slightly.